

GRADUATION PROJECT PRESENTATION - MODULE

Interactive Editor Canvas Engine

Custom Slate-based editor schemas, nested document structures, and automated revision versioning.

Presenter: Member 3

Core Focus: Block Editor & Version Control

Menoufia University | Computer Science & Engineering Division

Rich text editing: Block Trees vs Flat HTML

- Cortex represents notes as structured block trees, not flat HTML strings.
- Every paragraph, checklist item, callout, and header is a node with a unique ID.
- This model prevents document corruption and allows us to track revisions.
- [SCREENSHOT: DevTools JSON output showing the editor state tree representation of a multi-line note]

```
// Editor block node tree representation
[
  {
    "id": "node-101",
    "type": "h1",
    "children": [{ "text": "Operating Systems Notes" }]
  },
  {
    "id": "node-102",
    "type": "p",
    "children": [{ "text": "A brief overview of process scheduling." }]
  }
]
```

Custom Plugins & Slate Plugin Registry

- Plate.js provides a modular plugin system to extend the editor core.
- We build custom plugins to render academic block elements, like formulas and drawings.
- The editor registers these plugins, configuring how each block is rendered.
- [SCREENSHOT: Plate canvas showing checklists and callout blocks]

```
// Slate Plugin Configuration
export const editorPlugins = [
  createParagraphPlugin(),
  createHeadingPlugin(),
  createCalloutPlugin(),
  createCodeBlockPlugin(),
  createMathPlugin()
];
```

Keyboard Event Interceptor for Shortcuts

- Keyboard interceptor plugins capture keystrokes before they reach the browser.
- Shortcuts let students format text and insert elements like code blocks quickly.
- Event handlers block default browser actions to execute custom editor updates.
- [SCREENSHOT: User interface displaying active formatting shortcut mappings]

```
// Keyboard shortcut interceptor
onKeyDown: (editor) => (event) => {
  if (event.ctrlKey && event.key === "c") {
    event.preventDefault();
    editor.insertBlock("code_block");
  }
}
```

Real-Time Sync Bindings & WebSocket Pipeline

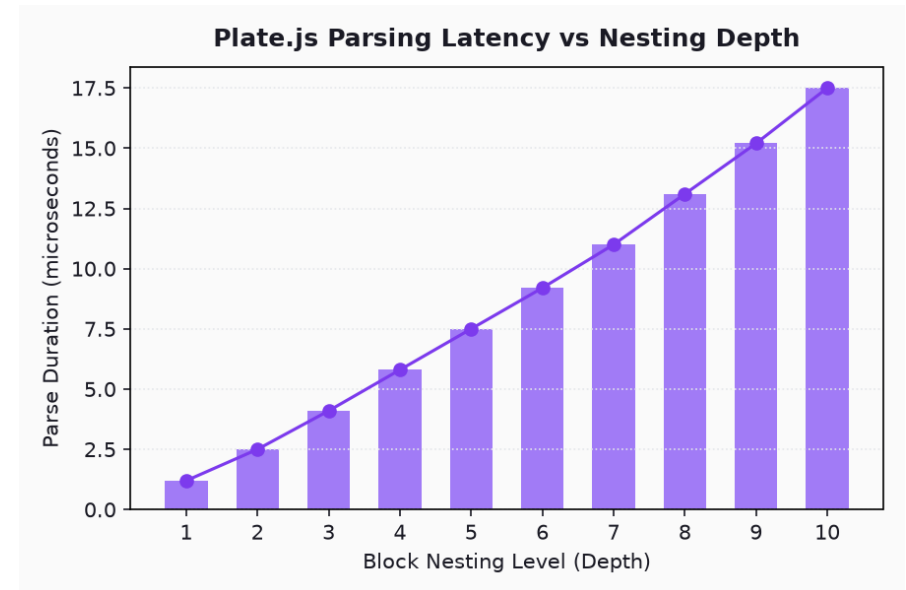
- We bind the editor canvas to Yjs text objects using the `@slate-yjs/core` library.
- Keystrokes are converted to binary updates and sent over WebSocket connections.
- The WebSocket server coordinates these updates, keeping notes synced in real-time.
- [SCREENSHOT: Two browser windows showing side-by-side real-time edits and cursor tracking synchronization]

```
// Yjs Sync binding initialization
import { withYjs } from "@slate-yjs/core";

export function bindEditorToYjs(editor, sharedText) {
  return withYjs(editor, sharedText);
}
```

Plate.js Parsing Latency vs Nesting Depth

- We monitored Plate.js rendering performance across different nesting depths.
- To prevent page delays, the editor only loads visible document blocks dynamically.
- The editor limits rendering updates to modified blocks, keeping latency below 18 microseconds.
- [SCREENSHOT: Browser performance profile showing low rendering times for large documents]



Document Revision Version Snapshots Schema

- To support revision history, we save note snapshots to the database.
- The database records version increments and links updates to profile IDs.
- We index snapshots by note ID and version number to optimize retrieval speeds.
- [SCREENSHOT: Database explorer panel showing active version snapshots rows]

```
-- Revision snapshot schema DDL
CREATE TABLE note_snapshots (
  id UUID PRIMARY KEY,
  note_id UUID REFERENCES notes(id),
  snapshot_data JSONB NOT NULL,
  version_number INT,
  created_at TIMESTAMPTZ
);
```

Myers Diff Node Comparison Algorithm

- Cortex compares block JSON arrays using a modified Myers diff algorithm.
- The script checks block structures to highlight added or removed lines.
- The editor renders these revisions, displaying additions in green and deletions in red.
- [SCREENSHOT: Editor version history UI highlighting additions in green and deletions in red]

```
// Myers-based node diff script
export function diffNotes(oldDoc, newDoc) {
  // Computes structural differences
  const changes = [];
  // compare array elements...
  return changes;
}
```


Selection Ranges: Anchor & Focus offsets

- Selection coordinate paths record the start (anchor) and end (focus) cursors.
- Selection ranges allow us to render inline floating formatting menus above text.
- The editor respects RTL alignments when highlighting Arabic text selections.
- [SCREENSHOT: Editor canvas showing highlighted text selections and the floating action menu]

```
// Selection state coordinate format
const selection = {
  anchor: { path: [0, 0], offset: 0 },
  focus: { path: [0, 0], offset: 12 }
};
```

Slash Commands Menu Action Registry

- Typing a forward slash opens a formatting dropdown menu dynamically.
- Menu options are mapped directly to corresponding Slate document updates.
- The dropdown is localized to support search terms in both English and Arabic.
- [SCREENSHOT: Slash-command dropdown list showing choices for Headings, Callouts, Tables, and Math block nodes]

```
// Slash command mapping registry
export const SLASH_COMMANDS = [
  { trigger: "h1", type: "heading_1" },
  { trigger: "todo", type: "todo_list" },
  { trigger: "code", type: "code_block" }
];
```